Cloud Cost Optimization

make cost visible, owned, then efficient

Watch the 4-min video

In one line: first make cost **visible and owned** (tags, allocation, unit economics), then remove waste (right-size, delete idle, turn off non-production), buy the steady baseline cheaply (reservations, savings plans), run interruptible work on **spot**, tier storage, and cut network egress; keep it with budgets and monthly reviews.

1 A $200,000 surprise

The bill arrives: **$200,000** this month, up 40%, and finance wants a third cut. Nobody can say which team, service or feature spends what. That's the normal state of a cloud account that grew fast. **FinOps**, the practice formalised by the FinOps Foundation, treats cost as an engineering metric with owners, in a loop of three phases.

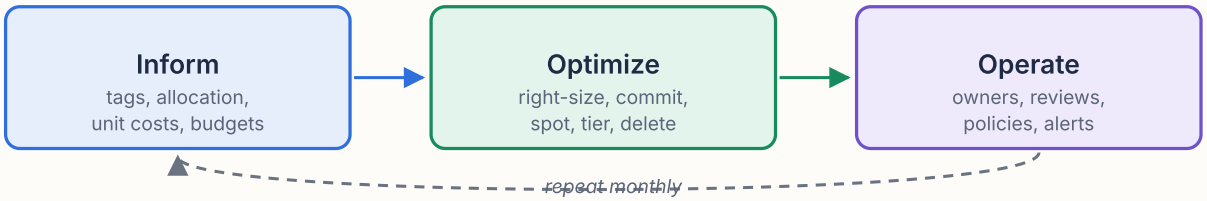


Figure 1. The FinOps lifecycle. You can't optimise what you can't attribute, and savings fade unless someone owns them.

2 Visibility first

- **Tag everything** with `team`, `service`, `environment` (and `cost-center`). Enforce tags with policy (Azure Policy, AWS tag policies / SCPs) so untagged resources can't be created. In Kubernetes, shared nodes need allocation by namespace or label: OpenCost or Kubecost split node cost by requests.
- **Showback** (each team sees its costs) usually comes before **chargeback** (costs hit team budgets).
- **Unit economics**: cost per request, per tenant, per active user, per order. A bill that grows with revenue is fine; a rising **cost per customer** is the real problem. If 10% traffic growth brings 40% cost growth, something is inefficient.

3 Right-size and clean up

Many VMs and pods run at 10% CPU because someone guessed big. Use 2–4 weeks of utilisation (p95 CPU and memory, not averages) to pick smaller instance sizes and lower Kubernetes requests (VPA recommendations, lesson 58). Move to newer generations, which are usually cheaper per unit of work (and ARM instances such as Graviton or Cobalt where the software runs on them). Look hard at databases: an oversized managed database is often the single biggest line item.

Then delete **zombies**: idle dev environments, unattached disks, unused public IPs, old snapshots and images, load balancers with no targets, forgotten test clusters. Each is small; together they are often 5–10% of a bill. Automate the hunt (Azure Advisor, AWS Trusted Advisor / Compute Optimizer) and give every resource an owner tag so someone can say "delete it".

4 Commit and discount

Pricing model	Typical discount vs on-demand	You give up	Use for
On-demand	—	nothing	spiky or unknown load
Savings plans (AWS, Azure)	roughly 15–65%	commit $/hour for 1 or 3 years; flexible on instance type	steady compute baseline
Reserved instances / reservations	up to ~70%+	commit to a type/region for 1 or 3 years	very stable fleets, databases
Spot / preemptible	up to ~90%	can be reclaimed: 2 min notice on AWS, ~30 s on Azure and GCP	batch, CI, stateless workers, some training

Commit only to the **baseline** you will certainly use: the load at 3 a.m. on a Sunday, after right-sizing (right-size first, or you lock in waste). Many teams aim to cover most of that baseline with commitments and keep the variable part on-demand.

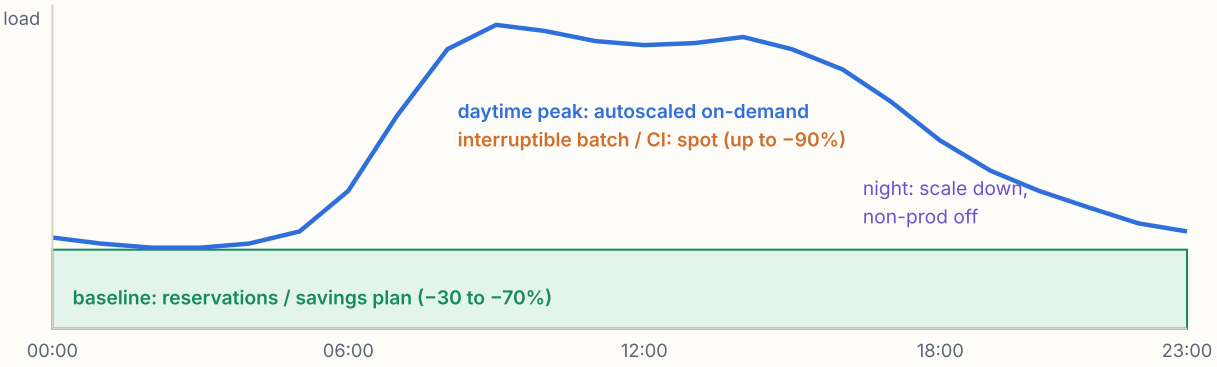


Figure 2. Layer the pricing models on the demand curve: commitments under the always-on baseline, on-demand autoscaling for the daily peak, spot for work that can be interrupted.

5 Follow demand, tier storage

Autoscaling (lesson 58) turns fixed capacity into capacity that follows the curve. **Non-production off-hours** is the easiest win: running dev/test 12 hours × 5 weekdays is 60 of 168 hours a week, about **64% less** than running all the time. Schedules, scale-to-zero, and ephemeral per-PR environments that delete themselves make it stick.

Storage tiering: object storage tiers (S3 Standard → Infrequent Access → Glacier tiers; Azure Hot → Cool → Cold → Archive) cost progressively less per GB stored but more per access, with minimum storage durations. Lifecycle rules move data by age and delete what you don't need (old logs, temporary exports, incomplete multipart uploads). Also check disk types (for example gp3 is cheaper than gp2 for the same size on AWS) and log retention, since observability data is a surprisingly large bill.

6 Network, budgets and making it stick



Figure 3. Costs that don't show up in architecture diagrams.

Data transfer out to the internet costs on the order of $0.05–0.09 per GB at list price on the big clouds; between regions and even between availability zones is also charged. Serve static content through a CDN, keep chatty services in the same zone where resilience allows, compress responses, and use private endpoints to avoid NAT gateway processing fees for traffic to cloud services. Lesson 65 puts numbers on these.

Set **budgets and anomaly alerts** per team and subscription/account, so a runaway job is found in a day, not at month end. Review the top cost drivers with each team monthly, track savings, and add guardrails (allowed instance types, maximum node counts, required tags) to the platform (lesson 70).

WORKED EXAMPLE: MOVING NIGHTLY BATCH TO SPOT

Nightly processing runs on **20 on-demand VMs for 6 hours**: 20 × 6 × 30 = 3,600 VM-hours a month. At an illustrative $0.40/hour that's about **$1,440**. The team moves it to a **spot node pool** spread over several instance types (so one type running out doesn't stop the job), makes each job **checkpoint progress every few minutes** to object storage, and keeps a small **on-demand fallback** pool that the scheduler uses if spot capacity is unavailable. When a node is reclaimed, the job gets the eviction notice, writes a final checkpoint if it can, and restarts elsewhere from the last checkpoint, losing at most a few minutes. With spot prices around 70–80% below on-demand and a little rework, the batch bill falls by about **70%**, to roughly $430 a month, and the job still finishes before morning.

COMMON MISTAKES

- Buying 3-year reservations before right-sizing, locking in waste.
- Running stateful or non-checkpointed jobs on spot and losing hours of work.
- Optimising the total bill without unit costs, so growth hides inefficiency.
- One-off clean-ups with no owners or budgets: costs creep back within months.

7 Key terms

FinOps: practice of making cloud cost visible, owned and optimised

Right-sizing: matching instance size or requests to real usage

Spot: spare capacity, deeply discounted, can be reclaimed

Unit economics: cost per request, tenant or user

Savings plan / reservation: discount for 1–3 year commitment

Showback / chargeback: report costs to teams / bill them

8 Interview questions

Q1. How would you cut a cloud bill by 30%? BASIC

Make cost visible with tags and unit economics; right-size and delete idle resources; use reservations or savings plans for steady load and spot for interruptible work; autoscale and shut down non-production at night; tier storage; and reduce egress.

Q2. What is unit economics? BASIC

Cost per request, tenant or user, so you can see whether efficiency improves as you grow, independent of total growth.

Q3. When is spot a bad fit? BASIC

For work that can't tolerate interruption without losing progress: stateful databases, long jobs without checkpoints, latency-critical services without fallback capacity.

Q4. Costs grew 40% while traffic grew 10%. Where do you look first? SCENARIO

Cost per unit by service and team: new or oversized resources, unattached storage, logs and data transfer growth, idle environments, and expired commitments now billed on-demand.

Q5. How much should you commit with reservations or savings plans? SCENARIO

Only the steady baseline after right-sizing, typically the minimum load over the period, leaving peaks to autoscaled on-demand and spot.

Q6. Why can moving to a cheaper storage tier increase the bill? TRICKY

Cooler tiers charge more per read and retrieval and have minimum storage durations, so frequently accessed or short-lived data costs more there.

Q7. Your Kubernetes cluster runs on shared nodes. How do you charge teams? TRICKY

Allocate node cost by each namespace's resource requests (or max of requests and usage) with a tool like OpenCost, plus shared costs split by a stated rule.

Next: SD-V-015 · Cost-Aware Architecture backendroadmap.com